

Hardware-Efficient Reinforcement Learning for Octopus CXL Memory Pooling

Pulak Mehrotra

March 19, 2026

Abstract

The current trace-driven Octopus simulator achieves fast per-step execution ($\sim 10,000$ steps/sec) but suffers from low end-to-end training throughput (~ 450 steps/sec) due to Python overhead and inefficient batching in the reinforcement learning loop. This results in severe underutilization of available GPU resources.

This proposal outlines a systems-driven redesign of the RL training pipeline, beginning with single-GPU saturation through batched rollouts and training, followed by a transition to a fully tensorized JAX-based simulator and eventual multi-GPU scaling. The goal is to reach Isaac-style performance regimes (10k–100k+ steps/sec) while preserving the fidelity of trace-driven cloud workloads.

1 Phase 0: Empirical Bottleneck Analysis

We profile the current Stable-Baselines3 (SB3)-based training pipeline on a single NVIDIA TITAN RTX.

Measured performance:

- Environment step time: ≈ 0.1 ms ($\sim 10,000$ steps/sec)
- Training throughput: ≈ 450 steps/sec
- GPU utilization: $< 10\%$

Key observation: The environment is already sufficiently fast and does not constitute the primary bottleneck. Instead, the training loop is dominated by Python overhead and inefficient batching.

Root causes:

- Single-environment rollout (no parallelism)
- Per-step Python control flow
- Small, infrequent GPU workloads (low duty cycle)

This results in a $\sim 10\text{--}20\times$ gap between observed throughput and hardware capability.

Conclusion: Multi-GPU scaling is premature. The immediate objective is to saturate a single GPU.

2 Phase 1: Batched Rollout and Training (Single GPU)

To eliminate Python overhead and improve GPU utilization, the training pipeline must operate on batched environment rollouts.

Vectorized environments. We run N environments in parallel:

$$s_t \in \mathbb{R}^{N \times d} \quad (1)$$

where $N \in [4, 32]$.

Batched policy inference. The policy network processes all observations simultaneously:

$$a_t = \pi_\theta(s_t) \quad (2)$$

This converts many small GPU calls into a single large kernel.

Decoupled rollout and training. Instead of updating every step, for SAC we set:

- `train_freq` = N (collect N steps before any gradient update)
- `gradient_steps` = N (match the collection rate)
- `batch_size` = 512 (larger GPU kernel per update)

For PPO, the effective rollout buffer grows automatically to $N \times n_steps$ and the minibatch size is scaled as $\min(N \times 64, 2048)$.

Precomputed episode events. A significant hidden cost is `env.reset()`: each episode start triggers $O(|VMs|)$ work to filter and sort VM arrivals for the randomly selected pod. With N parallel environments resetting independently, this cost multiplies. We address it by precomputing all event lists offline:

```
1 from octopus.data import precompute_pod_events
2
3 seeds = [base_seed + i for i in range(n_envs * max_episodes)]
4 precomputed = precompute_pod_events(trace_data, M, seeds)
5 # env.reset() now does a dict lookup instead of O(|VMs|) work
```

Listing 1: Precomputed event cache

Implementation. Three files were changed:

- `octopus/data.py` — `precompute_pod_events()` function
- `octopus/env.py` — `precomputed_events` parameter; `_load_precomputed()` fast path in `reset()`
- `scripts/train.py` — `--n-envs` argument; `SubprocVecEnv` when $N > 1$; scaled SAC/PPO hyperparameters

Measured results on a single NVIDIA TITAN RTX. Benchmarks run using `tests/napkin.math.py` (5,000-step runs, SAC, `net_arch=[128, 128]`, trace: AMS20PrdApp19).

Configuration	Steps/sec	Speedup
Baseline ($N = 1$, <code>tf=1</code> , <code>bs=256</code>)	46	1.0×
$N = 1$, <code>tf=16</code> , <code>gs=16</code> , <code>bs=512</code>	50	1.1×
$N = 4$, <code>SubprocVecEnv</code>	191	4.1×
$N = 8$, <code>SubprocVecEnv</code>	369	8.0×
$N = 16$, <code>SubprocVecEnv</code>	657	14.2×
$N = 32$, <code>SubprocVecEnv</code>	1221	26.5×

Reset time improves from ~ 120 ms to ~ 7 ms ($16.6\times$) with precomputed events.

Key observation. Increasing `train_freq` and `batch_size` alone (without additional environments) yields no measurable gain ($\sim 1\times$). The bottleneck is Python/CPU throughput, not GPU kernel size. Parallelism via `SubprocVecEnv` scales near-linearly up to $N = 32$.

Achieved improvement. Throughput increases from ~ 46 steps/sec to $\sim 1,220$ steps/sec ($\approx 26\times$) on a single TITAN RTX with $N = 32$ workers. The `--fast` flag in `train.py` now defaults to $N = 32$.

3 Phases 2–4: Empirical Assessment (JAX Rewrite)

Phases 2–4 originally proposed rewriting the environment in JAX with immutable PyTree state, tensorized dynamics, and zero-copy GPU trace execution. Before committing to this effort, we benchmarked the system at $60\times$ scale (all 10 available traces, ~ 1.1 M VMs total) to determine whether new bottlenecks would emerge that these phases address.

Measured results at $60\times$ scale. Benchmarks run using `tests/napkin_math.py --scale-test`.

Metric	Value	Notes
Total VMs (10 traces)	1,098,278	$32\times$ single trace
Trace load time (total)	~ 371 s	One-time startup cost
Precompute time per seed	115–244 ms	Scales with # VMs
<code>dealloc_events</code> RAM ($N=32$)	113 MB	Fixed by topology, not data size
Throughput, single trace ($N=32$)	1,225 steps/s	
Throughput, all 10 traces ($N=32$)	1,022 steps/s	16.6% IPC overhead

Assessment of each proposed phase.

Phase 2 (JAX PyTrees). JAX’s benefit is eliminating Python dispatch overhead on the per-step hot path. However, the `env` step is already 0.1 ms and is not the bottleneck — the bottleneck is `SubprocVecEnv` IPC. JAX cannot reduce IPC cost. Furthermore, the variable-length, sparse VM arrival structure is hostile to fixed-shape JAX arrays: a padded `active_vms` tensor would waste compute proportional to worst-case episode length. **Not recommended.**

Phase 3 (Tensorized dynamics). The masked-softmax allocation in `env.step()` is already 5 lines of NumPy taking < 0.02 ms. The remaining loop in `_process_departures_through()` is sparse and would require worst-case padding to tensorize. Potential saving: $< 1\%$. **Not recommended.**

Phase 4 (Zero-copy GPU trace). There is no CPU→GPU trace transfer to eliminate: the `env` is pure CPU/NumPy. Preloading the trace into GPU memory ($\mathcal{T} \in \mathbb{R}^{T \times n \times F}$) would require fully GPU-resident dynamics (Phase 2+3 prerequisite) and would consume several GB of VRAM per trace. **Not recommended.**

Actual bottleneck at $60\times$ scale: startup time. The dominant cost is trace deserialization: 6 minutes to load all 10 traces cold. This is a one-time cost per training run, but it is the largest single friction point. The correct intervention is lazy loading (each worker loads only its assigned trace) rather than a JAX rewrite.

4 Phase 5: Multi-GPU Scaling (3x TITAN RTX)

After saturating a single GPU, we scale via hierarchical parallelism.

```
1 import jax
2
3 vmapped_step = jax.vmap(step_fn, in_axes=(0, 0))
```

```
4 parallel_step = jax.pmap(vmapped_step, in_axes=(0, 0))
```

Listing 2: Hierarchical Parallelization

Important: Multi-GPU scaling is only effective once a single GPU is saturated. Otherwise, additional GPUs remain idle.

Expected scale:

- Single GPU: 10k–50k steps/sec
- 3 GPUs: 30k–150k steps/sec

5 Conclusion

The primary bottleneck in the current system is not simulation speed but inefficient training pipeline design. By restructuring the rollout and training loop to operate on large batches, we unlock an order-of-magnitude improvement on a single GPU. Only after achieving high utilization does it become meaningful to pursue JAX compilation and multi-GPU scaling.